Technical Documentation

Electrical Status Notification System

ระบบตรวจวัดและแจ้งเตือนสถานะอุปกรณ์ไฟฟ้าด้วย ESP32 และ INA219

โครงงานการศึกษาค้นคว้าอิสระ (IS)
โรงเรียนนครสวรรค์

1. Project Overview

1.1 วัตถุประสงค์

ระบบนี้ถูกออกแบบเพื่อแก้ปัญหาการตรวจสอบสถานะของอุปกรณ์ไฟฟ้า DC จากระยะไกล โดยระบบจะวัดค่าทางไฟฟ้าของอุปกรณ์ ได้แก่ แรงดันไฟฟ้า กระแสไฟฟ้า และกำลังไฟฟ้า

จากนั้น ESP32 จะนำข้อมูลจาก INA219 มาประมวลผลเพื่อประเมินว่าอุปกรณ์อยู่ในสถานะ ON หรือ OFF แล้วส่งข้อมูลผ่าน Wi-Fi ไปยัง Backend

Backend ทำหน้าที่เป็นศูนย์กลางในการรับข้อมูล จัดเก็บสถานะล่าสุดและประวัติการวัด ก่อนนำข้อมูลไปแสดงบนเว็บไซต์ และสร้างสรุปสำหรับการแจ้งเตือนผ่าน LINE

2. Scope

สิ่งที่ระบบทำ

```
Electrical Load -> Measurement -> INA219 -> ESP32 -> Wi-Fi ->
Backend API -> Storage -> Frontend -> User
```

รวมถึง: วัดแรงดัน, วัดกระแส, คำนวณ/อ่านค่ากำลัง, ตรวจจับสถานะ, รองรับ INA219 2 ตัว, ส่งข้อมูลผ่าน HTTP, เก็บสถานะล่าสุด, เก็บ historical readings, แสดงข้อมูลบนเว็บไซต์, ตรวจจับ stale data, แจ้งเตือนสรุปประจำวัน

สิ่งที่ไม่อยู่ใน **Scope**

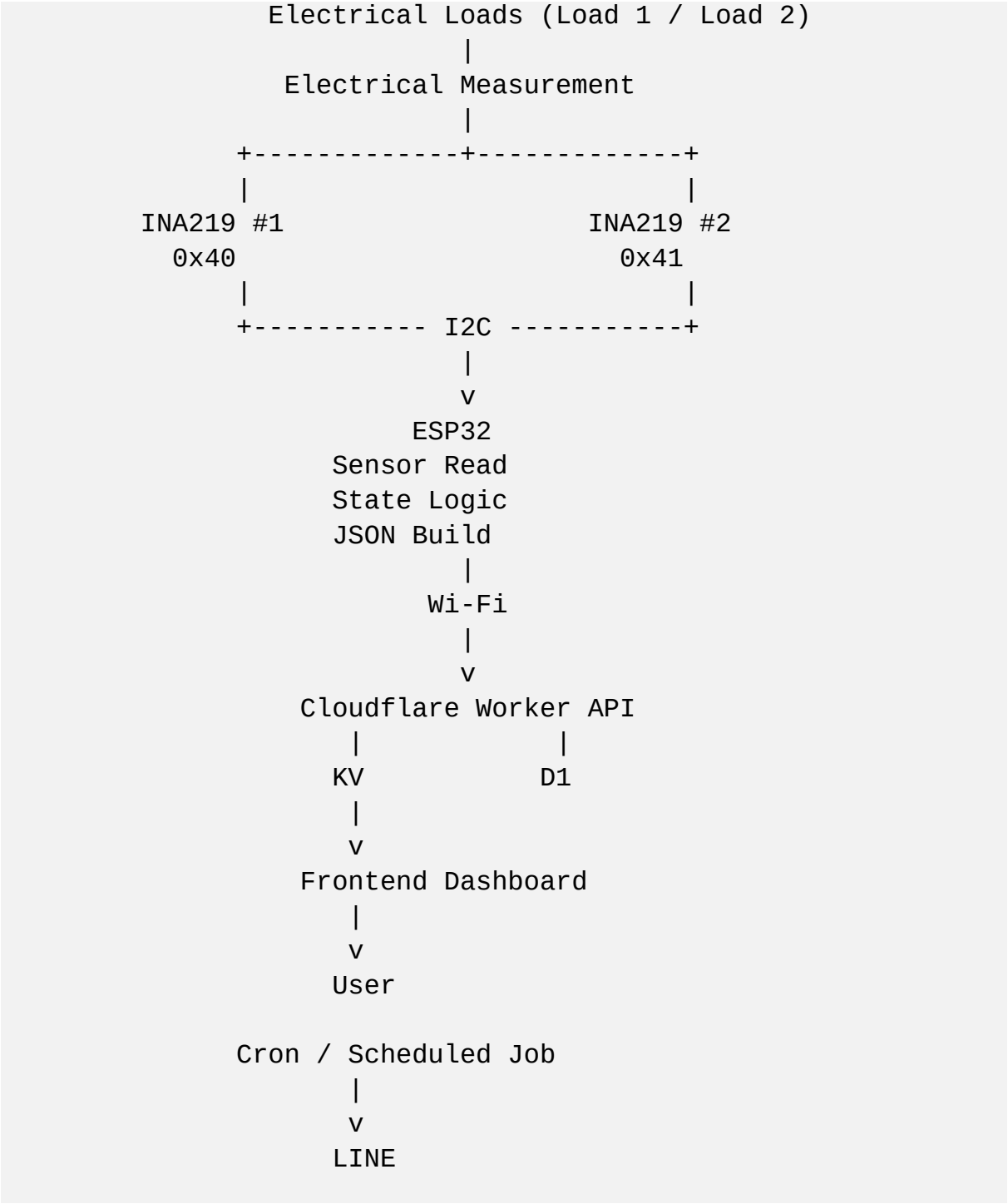
โครงการนี้ไม่ได้ออกแบบเพื่อ:

- ควบคุมเครื่องใช้ไฟฟ้าจากระยะไกล
- ป้องกัน over-current
- ป้องกัน short circuit
- เป็นระบบไฟฟ้าแรงดันสูง
- เป็นระบบ safety-certified
- ใช้แทน energy meter ที่ผ่านการรับรอง
- รับประกันความแม่นยำระดับเครื่องมือวัดมาตรฐาน

ขอบเขตของระบบคืออุปกรณ์ไฟฟ้ากระแสตรง (DC) ช่วง 3-9V

3. System Architecture

3.1 Overall Diagram



4. Hardware Architecture

4.1 Components

Component	หน้าที่
ESP32 DevKit	ประมวลผลและเชื่อมต่อ Wi-Fi

INA219 #1	วัด Load 1
INA219 #2	วัด Load 2
Expansion Board	เชื่อมต่อ hardware
Power Supply	จ่ายไฟให้ระบบ/โหลด
Electrical Loads	อุปกรณ์ที่ต้องการตรวจสอบ

5. INA219 Measurement Principle

INA219 ทำหน้าที่วัดค่าทางไฟฟ้าบนวงจร โดยค่าที่เกี่ยวข้องกับระบบคือ แรงดันไฟฟ้า (V) กระแสไฟฟ้า (I) และกำลังไฟฟ้า (P)

ความสัมพันธ์พื้นฐาน: กำลังไฟฟ้า = แรงดัน x กระแส (P = VI)

สำหรับการวัดกระแส หลักการของ current sensing ใช้แรงดันตกคร่อม shunt resistor:

```
V(shunt) = I x R(shunt)
ดังนั้น: I = V(shunt) / R(shunt)
```

ในระบบจริง เราใช้ค่าที่ได้จาก INA219/library เป็น input ให้ firmware

6. Dual INA219 Design

6.1 ทำไมใช้สองตัว

ต้องการแยกการวัดของอุปกรณ์สองชุด

```
                ESP32
                  |
                 I2C
            +-----+-----+
            |           |
        INA219       INA219
          #1           #2
       0x40         0x41
         |            |
       Load 1       Load 2
```

แต่ละ sensor มี identity ของตัวเอง

```
device_id -> ESP32 board
```

```
sensor_id -> INA219-01, INA219-02
```

7. I2C Architecture

7.1 Bus

```
ESP32 GPIO21 --- SDA ---+--- INA219 #1
                        +--- INA219 #2

ESP32 GPIO22 --- SCL ---+--- INA219 #1
                        +--- INA219 #2

ESP32 GND -------------+--- GND #1
                       +--- GND #2
```

ใช้ default ESP32 I2C ตามที่กำหนดใน firmware configuration

7.2 Addressing

```
INA219 #1: A0/A1 configuration -> 0x40
INA219 #2: A0/A1 configuration -> 0x41
```

I2C สามารถแชร์ bus ได้ แต่ address ต้องไม่ซ้ำกัน ถ้า INA219 ทั้งสองตัวเป็น 0x40 เหมือนกันจะเกิด address conflict ดังนั้นจึงกำหนดให้ต่างกันเป็น 0x40 และ 0x41

8. Circuit Diagram

แยก Signal Path กับ Power/Measurement Path เพื่อความชัดเจน

Signal

```
ESP32
 |
 +--- SDA ---+--- INA219 #1
 |           +--- INA219 #2
 |
 +--- SCL ---+--- INA219 #1
             +--- INA219 #2
```

Measurement

```
              INA219 #1
Supply + --- VIN+   VIN- --- Load 1 --- GND

              INA219 #2
Supply + --- VIN+   VIN- --- Load 2 --- GND
```

9. PCB Design Considerations

9.1 Signal Integrity

สิ่งที่ต้องพิจารณา: trace length, bus capacitance, pull-up resistors, SDA/SCL routing, crosstalk, EMI, ground reference, decoupling — SDA/SCL เป็น single-ended digital signal ไม่ใช่ differential pair

9.2 High-current Path

เส้นทางกระแสของ load ควรแยกจาก signal path

```
High-current path  ========================>
Signal path        ------------------------->
```

จุดประสงค์คือไม่ให้กระแสโหลดขนาดใหญ่ไหลผ่านพื้นที่ที่ไม่ควรเป็น current path และลดโอกาสที่ noise จะเข้ามากระทบ measurement

9.3 Decoupling

ควรมี capacitor ใกล้ supply ของ IC ตามคำแนะนำของ datasheet/reference design ที่ใช้จริง

```
VCC
 |
 +--- INA219
 |
 +--- Decoupling capacitor
          |
```

```
         GND
```

10. Firmware Architecture

แยก software เป็น module ตามแนวคิด Separation of Concerns

```
                    main.cpp
                       |
           +-----------+-----------+
           v           v           v
        sensor      Wi-Fi        API
           |           |           |
           v           v           v
       INA219      Network     HTTP/JSON
```

main.cpp ควบคุม flow, **sensor.cpp** รับผิดชอบ INA219, **wifi_manager** รับผิดชอบ Wi-Fi, **api_client** รับผิดชอบ communication กับ backend, **config** เก็บ configuration

11. Firmware Data Model

```
struct SensorReading {
    sensor_id;
    voltage_v;
    current_ma;
    power_mw;
    is_on;
    ok;
};
```

โครงสร้างแบ่งเป็น 3 กลุ่ม: Identity (sensor_id), Measurement (voltage, current, power), และ State (is_on, ok)

12. Firmware Execution Flow

```
setup()
  +-- Serial initialization
  +-- I2C initialization
```

```
  +-- INA219 #1 initialization
  +-- INA219 #2 initialization
  +-- Wi-Fi connection
          |
          v
        loop()
          |
          v
     Read Sensor #1
          |
          v
     Read Sensor #2
          |
          v
     Validate readings
          |
          v
     Determine state (hysteresis)
          |
          v
     Heartbeat / Change?
        No -> loop ต่อ
        Yes -> Build JSON -> Wi-Fi available? -> POST /update
-> loop
```

13. ON/OFF State Detection

ระบบใช้กระแสเป็นตัวแปรหลักในการบอกสถานะ โดยใช้หลักการ hysteresis (2 ขอบ) แทน threshold เดี่ยว เพื่อป้องกันสถานะกระพริบเมื่อกระแสแกว่งใกล้ค่า threshold

```
|I| > ON threshold   -> ON
|I| < OFF threshold  -> OFF
อยู่ระหว่างสองค่า     -> คงสถานะเดิมไว้
```

ตัวอย่าง logic ที่ใช้งานจริง:

```
float currentAbs = fabs(current_ma);

if (currentAbs > CURRENT_ON_THRESHOLD_MA) {
    is_on = true;
} else if (currentAbs < CURRENT_OFF_THRESHOLD_MA) {
    is_on = false;
```

```
} else {
    is_on = previousIsOn; // โซนกำกวม - คงสถานะเดิม
}
```

ค่าที่ใช้งานจริงในระบบ: ON threshold = 25 mA, OFF threshold = 15 mA — ค่าทั้งสองเป็นค่าประมาณเริ่มต้น ควรมีผลการทดลองสนับสนุนเพิ่มเติมจากการวัดโหลดจริง

14. ทำไมไม่ใช้ Voltage อย่างเดียว

สมมติอุปกรณ์ยังต่ออยู่กับ supply แต่ปิดอยู่: แรงดันยังคงใกล้เคียงแรงดัน supply แต่กระแสจะเข้าใกล้ 0 ดังนั้น voltage เพียงอย่างเดียวบอกไม่ได้ว่า load กำลังทำงานอยู่หรือไม่ ในขณะที่ current สามารถสะท้อนการใช้พลังงานของ load ได้โดยตรงกว่าในโจทย์นี้ ระบบจึงใช้กระแสเทียบกับ threshold เป็นตัวตัดสินสถานะ ON/OFF

15. API Architecture

Method	Endpoint	Function
POST	/update	ESP32 ส่งข้อมูล
GET	/status	Frontend ขอข้อมูล
GET	/health	ตรวจสอบ backend

16. /update

```
ESP32
 | POST /update (JSON)
 v
Worker
 +-- Parse
 +-- Validate
 +-- Update KV
 +-- Insert D1
 |
 v
Response
```

ตัวอย่าง schema:

```
{
```

```
  "device_id": "ESP32-01",
  "sensors": [
    {
      "sensor_id": "INA219-01",
      "voltage_v": 5.02,
      "current_ma": 120.4,
      "power_mw": 604.0,
      "is_on": true,
      "ok": true
    },
    {
      "sensor_id": "INA219-02",
      "voltage_v": 5.01,
      "current_ma": 0.2,
      "power_mw": 1.0,
      "is_on": false,
      "ok": true
    }
  ]
}
```

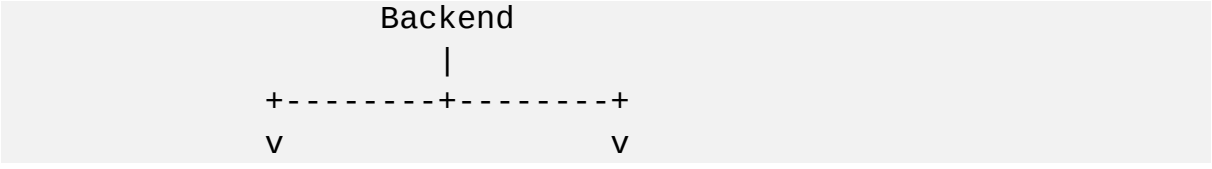
นี่เป็นตัวอย่าง schema ไม่ใช่ผลการวัดจริง

17. Backend Validation

```
Incoming Request -> Is JSON valid?
  No -> 400
  Yes -> Required fields?
           No -> 400
           Yes -> Validate sensors -> Store
```

ตรวจอย่างน้อย: JSON parse ได้หรือไม่, มี device_id หรือไม่, มี sensors หรือไม่, sensor ID ถูกต้องหรือไม่, numeric fields เป็นตัวเลขหรือไม่, state มีรูปแบบถูกต้องหรือไม่, sensor reading ใช้งานได้หรือไม่

18. Database Architecture



```
            KV                  D1
      Latest State          History
           |                  |
           |                  +-- readings
           |
           +-- current state
```

KV: เก็บข้อมูลที่ต้องการอ่านเป็นปัจจุบัน (device/sensor -> latest state)

D1: เก็บ historical data (Time 1 -> Reading, Time 2 -> Reading, ...)

19. Database Concept

ออกแบบเชิง relational:

```
devices
   |
   +---- sensors
            |
            +---- readings
```

```
Device
  +-- Sensor 1 -> Reading, Reading, Reading
  +-- Sensor 2 -> Reading, Reading, Reading
```

ข้อดีคือในอนาคตหากมี ESP32 ตัวที่สอง (ESP32 #2 พร้อม INA219 ของตัวเอง) ยัง สามารถขยาย architecture เดิมได้โดยไม่ต้องเปลี่ยนโครงสร้างตาราง

20. Frontend Architecture

```
            Backend
               |
               | GET /status
               v
         JavaScript fetchStatus()
                |
                v
             render()
      +---------+---------+
```

```
      v                    v
   Sensor OK          Sensor Invalid/Stale
      |                    |
      v                    v
  ON / OFF                 N/A
  Voltage
  Current
  Power
```

21. Frontend Polling

ระบบไม่ได้ให้ ESP32 ติดต่อ browser โดยตรง แต่ browser เรียก GET /status ไปที่ backend เพื่อรับข้อมูลล่าสุด แล้วทำซ้ำตาม polling interval ที่กำหนดไว้ในฝั่ง frontend

```
0 sec  -> GET /status
5 sec  -> GET /status
10 sec -> GET /status
15 sec -> GET /status
...
```

22. N/A State

OFF หมายถึง: ได้รับข้อมูลล่าสุด และ measurement ใช้งานได้ และ current ต่ำกว่า threshold

N/A หมายถึง: ไม่มีข้อมูลล่าสุดที่เชื่อถือได้ เช่น กรณี ESP32 หรือ Wi-Fi หรือ Backend มีปัญหาอย่างใดอย่างหนึ่ง

หาก frontend แสดง "OFF" ทั้งที่จริง ๆ ESP32 หายไปจากระบบ ผู้ใช้จะเข้าใจผิดว่าอุปกรณ์ปิดอยู่ ดังนั้นหลักการคือ: ไม่มีข้อมูลที่เชื่อถือได้ = N/A ไม่ใช่ = OFF ซึ่งปลอดภัยกว่าในเชิง information integrity ระบบใช้แนวคิดนี้เมื่อ ESP32 หลุดการเชื่อมต่อ

23. LINE Notification

```
Worker -> Scheduled Trigger -> Generate Summary -> LINE
Messaging API -> User
```

เนื้อหาที่แสดงในสรุป: Device, Sensor, Status, Measurement summary, Timestamp

เวลาที่ตั้งไว้: 16:10 น. เวลาไทย (เทียบเท่า 09:10 UTC)

24. Complete Data Flow

```
Electrical Load
       |
       | Electrical behavior
       v
INA219 Measurement
       |
       | I2C
       v
ESP32 (Read / Validate / Determine)
       |
       | HTTP / JSON
       v
Worker API
       |
       +----------------+
       v                v
      KV               D1
    Latest           History
       |
       | GET /status
       v
    Frontend
       |
       v
      User
```

25. Reliability / Failure Handling

Failure	ระบบควรทำอะไร
INA219 อ่านไม่ได้	ระบุ sensor ว่า invalid
ESP32 Wi-Fi หลุด	ไม่สามารถส่งข้อมูลใหม่
Backend ไม่ตอบ	ESP32 ต้องไม่ถือว่าการส่งสำเร็จ
Frontend ติดต่อ API ไม่ได้	แสดงสถานะที่เหมาะสม
Data stale	แสดง N/A
Sensor 1 fail	Sensor 2 ยังทำงานได้ตาม architecture

Sensor 2 fail	Sensor 1 ยังทำงานได้ตาม architecture

26. Security

```
ESP32 -- HTTPS --> Cloudflare Worker -- Validation -->
Database
```

หลักการ: ใช้ HTTPS สำหรับ communication, ไม่ใส่ credential จริงลง Git, secret เช่น LINE token อยู่ใน secret management (wrangler secret put), validate input ที่ backend, ไม่ให้ frontend เข้าถึง database โดยตรง

27. Configuration

```
Configuration
+-- Wi-Fi
+-- API endpoint
+-- Device ID
+-- Sensor IDs
+-- INA219 addresses
+-- Heartbeat interval
+-- ON/OFF threshold (hysteresis)
```

ไม่ commit secret จริงลง repository:

```
#define WIFI_SSID "YOUR_WIFI"
#define WIFI_PASSWORD "YOUR_PASSWORD"
#define API_ENDPOINT "YOUR_API_URL"
```

28. Testing Strategy

การทดสอบไม่ใช่แค่ "ลองเปิดแล้วมันติด" แต่แบ่งเป็น 5 ชั้น

```
              Testing
                 |
        +--------+--------+
        v        v        v
```

```
   Hardware  Firmware  Backend
       |        |        |
       +--------+--------+
                v
             Frontend
                |
                v
           End-to-End
```

29. Hardware Test

Test A — INA219 detection: ต่อ INA219 -> ESP32 startup -> ตรวจสอบการ initialize คาดหวังว่า INA219 #1 และ #2 detected ทั้งคู่

Test B — I2C address: ใช้ I2C scanner ตรวจสอบ คาดหวังเห็นทั้ง 0x40 และ 0x41 บันทึก Serial output ไว้เป็นหลักฐาน

30. Measurement Validation

ถ้ามี multimeter/reference meter ให้วัดเทียบกันระหว่าง INA219 กับเครื่องมืออ้างอิงที่ต่อกับ load เดียวกัน

Test	Reference	INA219	Error
Voltage 1			
Voltage 2			
Current 1			
Current 2			

คำนวณค่าความคลาดเคลื่อนร้อยละ = |ค่าที่วัดได้ - ค่าอ้างอิง| / ค่าอ้างอิง x 100

31. State Detection Test

ทดสอบหลายกรณีเรียงลำดับ: OFF -> Small load -> Normal ON -> OFF

Condition	Current	Expected State	Actual
OFF		OFF	

ON		ON	
Near threshold			
OFF again		OFF	

การบันทึกผลแบบนี้ทำให้ threshold ที่เลือกใช้มีหลักฐานรองรับ

32. Communication Test

```
ESP32 -> Wi-Fi -> Worker
```

ทดสอบ: Wi-Fi ปกติ, Wi-Fi หลุด, Wi-Fi กลับมา, Backend ปกติ, Backend unavailable

33. API Test

ทดสอบ POST /update ด้วยกรณี: Valid JSON, Invalid JSON, Missing device_id, Missing sensor, Wrong data type, Unknown sensor — บันทึก Expected response จากระบบจริงทุกกรณี

34. Frontend Test

Condition	Expected
Sensor ON	ON
Sensor OFF	OFF
Sensor invalid	N/A/invalid state
No API	appropriate error state
Stale data	N/A
Two sensors	แสดงแยกกัน

35. End-to-End Test

การทดสอบทั้ง chain คือหลักฐานที่มีน้ำหนักที่สุดสำหรับการนำเสนอ

```
Physical Load -> INA219 -> ESP32 -> Wi-Fi -> Worker -> KV/D1
-> Website
```

เปลี่ยนสถานะจริงจาก OFF เป็น ON แล้วเก็บหลักฐานทุกจุด: Physical device, Serial Monitor, API request, Worker response, D1 record, Website

36. Performance Measurement

ค่าที่ควรวัดแทนการอ้าง "real-time" แบบไม่มีตัวเลขรองรับ: เวลาที่ใช้อ่าน sensor (Sensor -> ESP32), network latency (ESP32 -> Backend), polling delay (Backend -> Frontend), และ end-to-end latency รวม (Physical event -> Sensor detection -> ESP32 -> Backend -> Frontend)

ควรเก็บค่า Minimum, Average, Maximum ของแต่ละช่วง ค่า configuration ของระบบ (เช่น sampling 1s, upload 10s, polling 5s) เป็นค่าที่ตั้งไว้ ไม่ใช่ end-to-end latency ที่วัดได้จริง จนกว่าจะมีการทดลองวัดค่าจริงรองรับ

37. Limitations

Measurement: accuracy ขึ้นกับ sensor/configuration, load ที่มี current ต่ำมากอาจอยู่ใกล้ threshold, threshold อาจแตกต่างกันตามชนิด load

Network: ระบบต้องพึ่งพา Wi-Fi, Internet outage ทำให้ cloud update ไม่ได้

Cloud: ระบบพึ่งพา external cloud services, API/service availability ขึ้นกับ provider

Hardware: I2C signal integrity ขึ้นกับ PCB/wiring, electrical load ต้องอยู่ในขอบเขตที่ hardware รองรับ

38. Future Improvements

รายการต่อไปนี้ยังไม่ได้ implement เป็นเป้าหมายสำหรับการพัฒนาต่อในอนาคต:

```
Future Work
+-- Historical graph
+-- Multiple ESP32 devices
+-- Better threshold calibration
+-- Automatic threshold learning
```

```
+-- OTA firmware update
+-- Local buffering when offline
+-- Authentication for API
+-- Custom PCB revision
```

39. Summary Diagrams

สรุปเป็น 4 แผนภาพหลักของระบบ

แผนภาพที่ 1 — Overall

```
Load -> INA219 x2 -> (I2C) -> ESP32 -> (Wi-Fi) -> Worker ->
KV/D1 -> Frontend -> User
```

แผนภาพที่ 2 — Circuit

```
ESP32
 +-- SDA -- INA219 #1 (0x40)
 |        + INA219 #2 (0x41)
 +-- SCL -- INA219 #1
          + INA219 #2

Load -- INA219 -- Supply
```

แผนภาพที่ 3 — Backend

```
ESP32 -- POST --> Worker
                    +-- KV
                    +-- D1
                         |
                         v
                     Frontend -> User

Cron -> LINE
```

แผนภาพที่ 4 — Frontend

```
GET /status -> JavaScript -> Render
```

```
                                +-- ON/OFF
                                +-- N/A
```